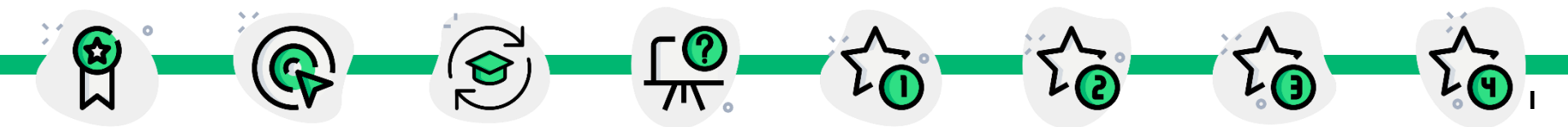


「2025년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2026.06.08

14주차



리스트

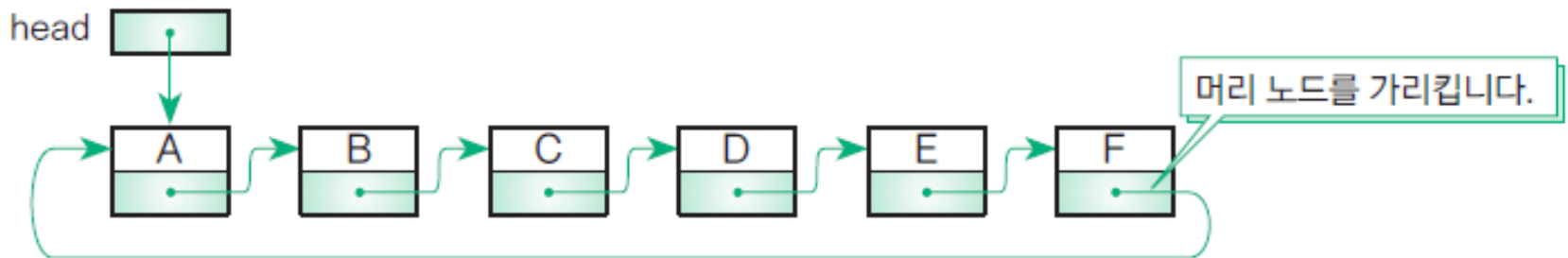


08-4 원형 이중 연결 리스트

원형 리스트 알아보기 - (1)

■ 원형 리스트 circular list

- 선형 리스트의 꼬리 노드가 머리 노드를 가리킴
- 고리 모양으로 나열된 데이터를 저장할 때 알맞은 자료구조
- 원형 리스트와 선형 리스트의 차이점은 꼬리 노드의 다음 노드를 가리키는 포인터가 널(NULL)이 아니라 머리 노드의 포인터값이라는 점
- 원형 리스트는 선형 리스트에서 사용했던 것과 같은 자료형을 사용할 수 있음



[그림 8-18] 원형 리스트



08-4 원형 이중 연결 리스트

원형 리스트 알아보기 - (2)

- 빈 원형 리스트를 판단하는 방법
 - 노드가 없는(비어 있는) 원형 리스트인지 판단하려면 오른쪽 식을 사용
- 노드가 1개인 원형 리스트를 판단하는 방법
 - 노드가 1개라면 머리 노드의 다음 포인터는 자기 자신인 머리 노드를 가리킴
- 포인터가 머리 노드를 가리키는지 판단하는 방법
 - Node*형 변수 p가 리스트에 있는 어떤 노드를 가리키는 경우 p가 가리키고 있는 노드가 원형 리스트의 머리 노드인지 판단하려면 오른쪽 식을 사용
- 포인터가 꼬리 노드를 가리키는지 판단하는 방법
 - Node*형 변수 p가 리스트에 있는 어떤 노드를 가리키는 경우 p가 가리키고 있는 노드가 원형 리스트의 꼬리 노드인지 판단하려면 오른쪽 식을 사용

```
list->head == NULL    // 원형 리스트가 비어 있는지 확인
```

```
list->head->next == list->head    // 노드가 1개인지 확인
```

```
p == list->head    // p가 가리키는 노드가 머리 노드인지 확인
```

```
p->next == list->head    // p가 가리키는 노드가 꼬리 노드인지 확인
```

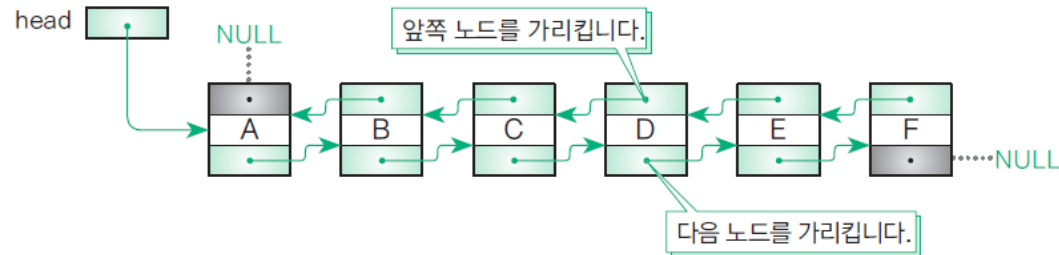


08-4 원형 이중 연결 리스트

이중 연결 리스트 알아보기 - (1)

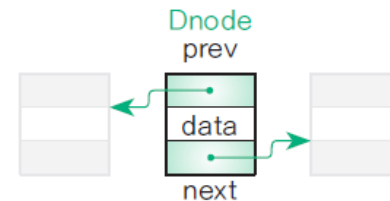
■ 이중 연결 리스트 doubly linked list

- 선형 리스트의 가장 큰 단점은 다음 노드는 찾기 쉽지만 앞쪽의 노드를 찾으려면 비용이 든다는 점을 개선한 자료구조
- 각 노드에는 다음 노드에 대한 포인터와 앞쪽의 노드에 대한 포인터가 주어짐
- 그림 8-20처럼 3개의 멤버가 있는 구조체를 사용



[그림 8-19] 이중 연결 리스트

```
typedef struct __node {  
    Member    data;    // 데이터  
    struct __node *prev; // 앞쪽 노드에 대한 포인터  
    struct __node *next; // 다음 노드에 대한 포인터  
} Dnode;
```



[그림 8-20] 이중 연결 리스트의 노드 구성



08-4 원형 이중 연결 리스트

이중 연결 리스트 알아보기 - (2)

- 포인터가 머리 노드를 가리키는지 판단하는 방법
 - 변수 p 가 가리키는 노드가 이중 연결 리스트의 머리 노드인지 판단하려면 오른쪽 식을 사용
- 포인터가 꼬리 노드를 가리키는지 판단하는 방법
 - p 가 가리키고 있는 노드가 꼬리 노드인지 판단하려면 오른쪽 식을 사용

```
p == list->head    // p가 가리키는 노드가 머리 노드인지 확인  
p->prev == NULL    // p가 가리키는 노드가 머리 노드인지 확인
```

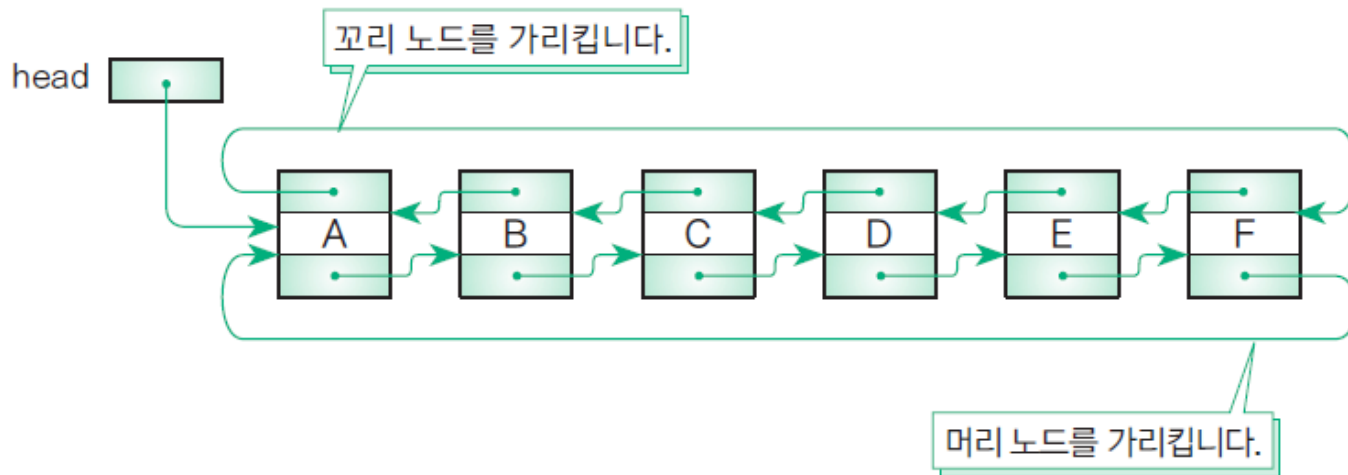
```
p->next == NULL    // p가 가리키는 노드가 꼬리 노드인지 확인
```



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (1)

- 원형 이중 연결 리스트 circular doubly linked list
 - 앞에서 공부한 두 가지의 개념을 합함



[그림 8-21] 원형 이중 연결 리스트



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (2)

■ 실습 8-7 - (1)

- 원형 이중 연결 리스트를 구현하는 프로그램(헤더)

Do it! 실습 8-7

• 완성 파일 chap08/CircDbllLinkedList.h

```
01  /* 원형 이중 연결 리스트(헤더) */
02  #ifndef __CircDbllLinkedList
03  #define __CircDbllLinkedList
04  #include "Member.h"
05
06  /*--- 노드 ---*/
07  typedef struct __node {
08      Member data;           // 데이터
09      struct __node *prev;   // 앞쪽 노드에 대한 포인터
10      struct __node *next;   // 다음 노드에 대한 포인터
11  } Dnode;
12
13  /*--- 원형 이중 연결 리스트 ---*/
14  typedef struct {
15      Dnode *head;           // 머리의 더미 노드에 대한 포인터
16      Dnode *crnt;           // 선택한 노드에 대한 포인터
17  } Dlist;
```

실습 10-1에서 작성



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (2)

■ 실습 8-7 - (1)

- 원형 이중 연결 리스트를 구현하는 프로그램(헤더)

```
19  /*--- 리스트를 초기화 ---*/
20  void Initialize(Dlist *list);
21
22  /*--- 선택한 노드의 데이터를 출력 ---*/
23  void PrintCurrent(const Dlist *list);
24
25  /*--- 선택한 노드의 데이터를 출력(줄 바꿈 문자 추가) ---*/
26  void PrintLnCurrent(const Dlist *list);
```

```
28  /*--- compare 함수로 x와 일치하는 노드를 검색 ---*/
29  Dnode *Search(Dlist *list, const Member *x,
30              int compare(const Member *x, const Member *y));
31
32  /*--- 모든 노드의 데이터를 리스트 순서대로 출력 ---*/
33  void Print(const Dlist *list);
34
35  /*--- 모든 노드의 데이터를 리스트 순서와 역순으로 출력 ---*/
36  void PrintReverse(const Dlist *list);
37
38  /*--- 선택한 노드의 다음으로 진행 ---*/
39  int Next(Dlist *list);
40
41  /*--- 선택한 노드의 앞쪽으로 진행 ---*/
42  int Prev(Dlist *list);
43
```



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (3)

■ 실습 8-7 - (2)

```
44  /*--- p가 가리키는 노드 바로 뒤에 노드를 삽입 ---*/
45  void InsertAfter(Dlist *list, Dnode *p, const Member *x);
46
47  /*--- 머리에 노드를 삽입 ---*/
48  void InsertFront(Dlist *list, const Member *x);
49
50  /*--- 꼬리에 노드를 삽입 ---*/
51  void InsertRear(Dlist *list, const Member *x);
52
53  /*--- p가 가리키는 노드를 삭제 ---*/
54  void Remove(Dlist *list, Dnode *p);
```

```
56  /*--- 머리 노드를 삭제 ---*/
57  void RemoveFront(Dlist *list);
58
59  /*--- 꼬리 노드를 삭제 ---*/
60  void RemoveRear(Dlist *list);
61
62  /*--- 선택한 노드를 삭제 ---*/
63  void RemoveCurrent(Dlist *list);
64
65  /*--- 모든 노드를 삭제 ---*/
66  void Clear(Dlist *list);
```

```
68  /*--- 원형 이중 연결 리스트 종료 ---*/
69  void Terminate(Dlist *list);
70  #endif
```



08-4 원형 이중 연결 리스트

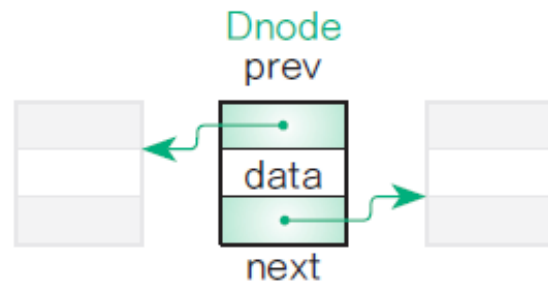
원형 이중 연결 리스트 만들기 - (4)

■ 노드를 나타내는 구조체 Dnode

- Dnode는 그림8-20의 이중 연결 리스트의 노드에서 사용
 - data... 데이터
 - prev... 앞쪽 노드에 대한 포인터
 - next... 다음 노드에 대한 포인터

■ 원형 이중 연결 리스트를 관리하는 구조체 Dlist

- 선형 리스트의 List와 마찬가지로 머리 노드에 대한 포인터와 선택한 노드에 대한 포인터를 가지고 있음



[그림 8-22] 원형 이중 연결 리스트의 노드 구성



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (5)

■ 노드를 생성하는 AllocDnode 함수

- Dnode형 객체를 생성하고 해당 객체의 포인터를 반환하는 함수

■ 노드의 멤버값을 설정하는 SetDnode 함수

- Dnode형 객체의 멤버값을 설정
- 첫 번째 매개변수 n에 전달받은 Dnode형 객체 포인터를 통해 멤버값을 설정
- 객체 멤버인 data, prev, next에 두 번째 매개변수가 가리키는 객체의 값, 세 번째 매개변수와 네 번째 매개변수의 포인터값을 대입

Do it! 실습 8-8[A]

• 완성 파일 chap08/CircDblLinkedList.c

```
01  /* 원형 이중 연결 리스트(소스) */
02  #include <stdio.h>
03  #include <stdlib.h>
04  #include "Member.h" — 실습 10-1에서 작성
05  #include "CircDblLinkedList.h"
06
07  /*--- 1개의 노드를 동적으로 생성 ---*/
08  static Dnode *AllocDNode (void)
09  {
10      return calloc(1, sizeof(Dnode));
11  }
12
```

```
13  /*--- 노드의 각 멤버값을 설정 ----*/
14  static void SetDNode (Dnode *n, const Member *x, const Dnode *prev, const Dnode *next)
15  {
16      n->data = *x;                      // 데이터
17      n->prev = prev;                    // 앞쪽 노드에 대한 포인터
18      n->next = next;                    // 다음 노드에 대한 포인터
19  }
```



08-4 원형 이중 연결 리스트

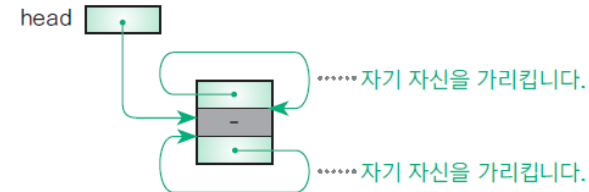
원형 이중 연결 리스트 만들기 - (6)

■ 원형 이중 연결 리스트를 초기화하는 Initialize 함수

- 텅 비어 있는 상태의 원형 이중 연결 리스트를 만드는 함수
- 리스트의 머리 부분에 더미 노드가 만들어짐(노드의 삽입, 삭제를 원만히 수행하기 위해 필요)
- 3개 포인터가 가리키는 대상 모두를, 리스트의 앞쪽에 있는 더미라고 함
 - 머리 포인터 list->head가 가리키는 대상
 - 더미 노드의 앞쪽 포인터 list->head->prev가 가리키는 대상
 - 더미 노드의 다음 포인터 list->head->next가 가리키는 대상

```
27  /*--- 리스트를 초기화 ---*/
28  void Initialize (Dlist *list)
29  {
30      Dnode *dummyNode = AllocDNode();    // 더미 노드 생성
31      list->head = list->crnt = dummyNode;
32      dummyNode->prev = dummyNode->next = dummyNode;
33  }
```

더미 노드만 있는 상태입니다.



[그림 8-23] 원형 이중 연결 리스트를 초기화한 상태



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (7)

- 리스트가 비어 있는지 검사하는 **IsEmpty** 함수
 - 더미 노드의 뒤쪽 포인터 `list->head->next`가 자신의 더미 노드인 `list->head`를 가리키는지를 판단
 - 함수의 반환값은 리스트가 비어 있는 경우에는 1(아닌 경우에는 0)
- 선택한 노드의 데이터를 출력하는 **PrintCurrent / PrintLnCurrent** 함수
 - **PrintCurrent** 함수는 `list->crnt`가 가리키는 노드의 데이터를 **PrintMember** 함수를 이용해 출력
 - 리스트가 비어 있는 경우에는 '선택한 노드가 없습니다.'라는 메시지를 출력
 - **PrintLnCurrent** 함수는 출력한 후의 개행 문자를 출력

```
21  /*--- 리스트가 비어 있는지 검사 ---*/
22  static int IsEmpty (const Dlist *list)
23  {
24      return list->head->next == list->head;
25  }
```

```
35  /*--- 선택한 노드의 데이터를 출력 ---*/
36  void PrintCurrent (const Dlist *list)
37  {
38      if(IsEmpty(list))
39          printf("선택한 노드가 없습니다.");
40      else
41          PrintMember(&list->crnt->data);
42  }
43
44  /*--- 선택한 노드의 데이터를 출력(줄 바꿈 문자 추가) ---*/
45  void PrintLnCurrent (const Dlist *list)
46  {
47      PrintCurrent(list);
48      putchar('\n');
49  }
```

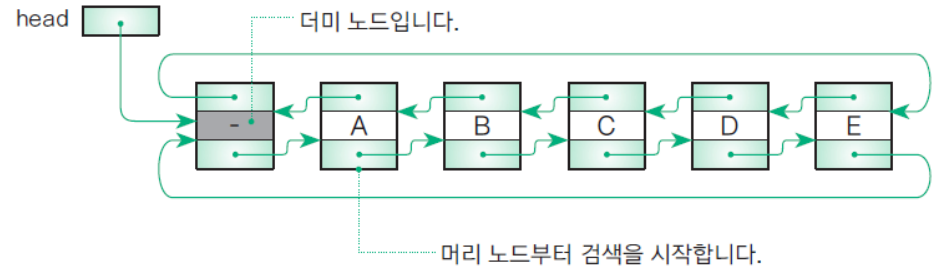


08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (8)

■ 노드를 검색하는 Search 함수 - (1)

- 머리 노드부터 뒤쪽 포인터를 이용해 순서대로 스캔
- 머리 노드는 더미 노드가 아니라 더미 노드의 다음 노드
- 그림 8-24에서 머리 포인터 list->head가 가리키는 더미 노드의 뒤쪽 포인터가 가리키는 노드 A가 진짜 머리 노드
- 따라서 검색을 시작하는 노드의 위치는 list->head가 가리키는 노드가 아닌 list->head->next가 가리키는 노드



[그림 8-24] 노드 검색의 시작 위치

Do it! 실습 8-8[B]

• 완성 파일 chap08/CircDbllinkedList.c

```
01  /*--- compare 함수로 x와 일치하는 노드를 검색 ---*/
02  Dnode *Search (Dlist *list, const Member *x,
03                int compare (const Member *x, const Member *y))
04  {
05      Dnode *ptr = list->head->next;
06      while(ptr != list->head)
07          if(compare(&ptr->data, x) == 0)
08              list->crnt = ptr;
09              return ptr;      // 검색 성공
10
11      ptr = ptr->next;
12
13      return NULL;             // 검색 실패
14
```

(실습 8-8[C]에서 계속)



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (9)

■ 노드를 검색하는 Search 함수 - (2)

- Node *형의 포인터 a, b, c, d, e가 각각 노드 A, 노드 B, ..., 노드 E를 순서대로 가리키는 경우 각 노드를 가리키는 식은 오른쪽 표와 같음(각각의 식은 노드 자신을 의미)
- Search 함수는 while 문으로 노드를 하나씩 스캔하는 과정에서 비교 함수인 compare 함수를 사용
 - compare 함수로 비교한 결과가 0이면 검색 성공이며 찾은 노드에 대한 포인터인 ptr을 반환
 - 이때 cmt는 찾은 노드(ptr)를 가리키도록 설정
- 노드를 찾지 못하고 한 바퀴 돌아 다시 더미 노드로 돌아오면(ptr이 head와 같으면) 검색에 실패한 것이므로 while 문을 종료하고 널(NULL)을 반환

더미 노드	head	e->next	d->next->next	a->prev	b->prev->prev
노드 A	a	headnext	e->next->next	b->prev	c->prev->prev
노드 B	b	a->next	head->next->next	c->prev	d->prev->prev
노드 C	c	b->next	a->next->next	d->prev	e->prev->prev
노드 D	d	c->next	b->next->next	e->prev	head->prev->prev
노드 E	e	d->next	c->next->next	head->prev	a->prev->prev

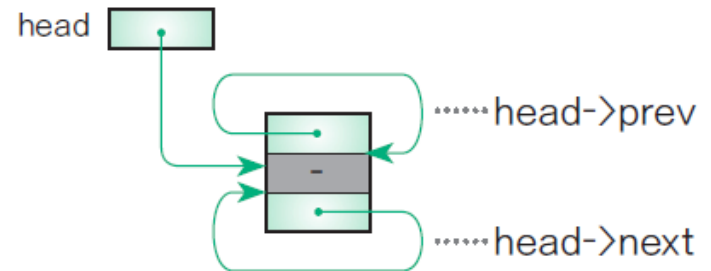


08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (10)

■ 노드를 검색하는 Search 함수 - (3)

- 빈 리스트를 검색하는 경우라 가정하고 이 함수가 정말 검색에 실패하는지(널(NULL)을 반환하는지) 그림 8-25를 통해 확인
- Ptr에 대입하는 list->head->next 값은 더미 노드에 대한 포인터
- 다시 말해 ptr은 list->head와 같은 값
- 그러면 while 문의 제어식 ptr != list->head가 성립되지 않기 때문에 while 문은 실행되지 않고 바로 널(NULL)을 반환하며 함수가 종료



[그림 8-25] 빈 원형 이중 연결 리스트를 검색하는 경우



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (11)

■ 모든 노드를 순서대로 출력하는 Print 함수

- list->head->next부터 스캔하기 시작해 뒤쪽 포인터를 찾아가며 각 노드의 데이터를 출력
 - a는 ①→②→③...의 순서로 포인터를 찾아가
- 다시 head로 돌아오면 스캔을 종료
 - ⑥을 찾아가면 다시 더미 노드로 돌아온 것과 같으므로 스캔을 종료

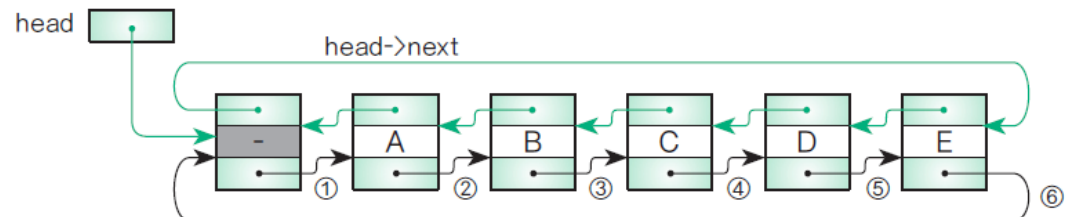
Do it! 실습 8-8[C]

• 완성 파일 chap08/CircDbLinkedList

```
01  /*--- 모든 노드의 데이터를 리스트 순서대로 출력 ---*/
02  void Print (const Dlist *list)
03  {
04      if(IsEmpty(list))
05          puts("노드가 없습니다.");
06      else {
07          Dnode *ptr = list->head->next;
08          puts("【 모두 보기 】");
09          while(ptr != list->head) {
10              PrintLnMember(&ptr->data);
11              ptr = ptr->next;          // 다음 노드 선택
12          }
13      }
14  }
15
```

(실습 8-8[D]에서 계속)

a 머리부터 모든 노드를 스캔합니다.





08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (12)

■ 모든 노드를 거꾸로 출력하는 PrintReverse 함수

- list->head->prev부터 스캔하기 시작해 앞쪽 포인터를 **b**아가며 각 노드의 데이터를 출력
 - 는 ①→②→③...의 순서로 포인터를 찾아감
- 다시 head로 돌아오면 스캔을 종료
 - ⑥을 찾아가면 다시 더미 노드로 돌아온 것과 같으므로 스캔을 종료

Do it! 실습 8-8[D]

• 완성 파일 chap08/CircDblLinkedList.c

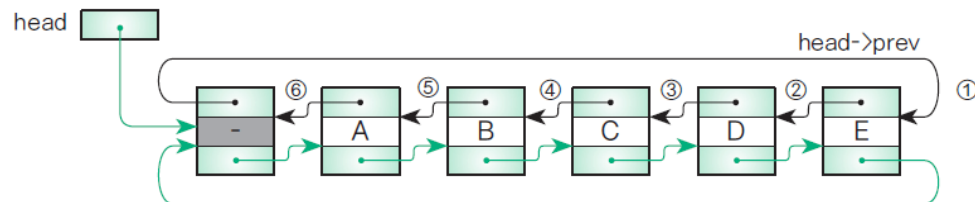
```

01  /*--- 모든 노드의 데이터를 리스트 순서의 역순으로 출력 ---*/
02  void PrintReverse(const Dlist* list)
03  {
04      if (IsEmpty(list))
05          puts("노드가 없습니다.");
06      else {
07          Dnode* ptr = list->head->prev;
08
09          puts("【 모두 보기 】");
10          while (ptr != list->head) {
11              PrintLnMember(&ptr->data);
12              ptr = ptr->prev;          // 앞쪽 노드 선택
13          }
14      }
15  }
16

```

(실습 8-8[E]에서 계속)

b 꼬리부터 모든 노드를 스캔합니다.



[그림 8-26] 모든 노드를 스캔하는 과정



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (13)

- 선택한 노드의 다음으로 진행시키는 **Next** 함수
 - 리스트가 비어 있지 않고 선택한 노드의 다음 노드가 있는 경우에만 동작
 - 선택한 노드가 다음 노드로 진행하는 데 성공하면 1, 실패하면 0을 반환
- 선택한 노드의 앞쪽으로 진행시키는 **Prev** 함수
 - 리스트가 비어 있지 않고 선택한 노드의 앞쪽 노드가 있는 경우에만 동작
 - 선택한 노드가 앞쪽 노드로 되돌아가는 데 성공하면 1, 실패하면 0을 반환

Do it! 실습 8-8[E]

• 완성 파일 chap08/CircDblLinkedList.

```
01  /*--- 선택한 노드를 다음으로 진행 ---*/
02  int Next(Dlist* list)
03  {
04      if (IsEmpty(list) || list->crnt->next == list->head)
05          return 0;                // 진행할 수 없음
06      list->crnt = list->crnt->next;
07      return 1;
08  }
09
10  /*--- 선택한 노드를 앞쪽으로 진행 ---*/
11  int Prev(Dlist* list)
12  {
13      if (IsEmpty(list) || list->crnt->prev == list->head)
14          return 0;                // 되돌아갈 수 없음
15      list->crnt = list->crnt->prev;
16      return 1;
17  }
18
```

(실습 8-8[F]에서 계속)



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (14)

■ 바로 다음에 노드를 삽입하는 InsertAfter 함수 - (1)

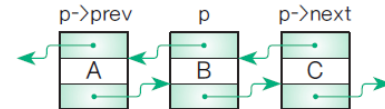
- 포인터 p가 가리키는 노드의 바로 다음에 노드를 삽입
- 그림 8-27에서 노드를 삽입한 위치는 p가 가리키는 노드와 p->next가 가리키는 노드의 사이
 - 1. 새로 삽입할 노드 D를 만들고 만든 노드의 앞쪽 포인터가 가리키는 노드는 B, 뒤쪽 포인터가 가리키는 노드는 C로 설정
 - 2. 노드 B의 뒤쪽 포인터 p->next와 노드 C의 앞쪽 포인터 p->next->prev 모두 새로 삽입한 노드를 가리키도록 업데이트
 - 3. 선택한 포인터 list->crnt가 삽입한 노드를 가리키도록 업데이트

Do it! 실습 8-8[F]

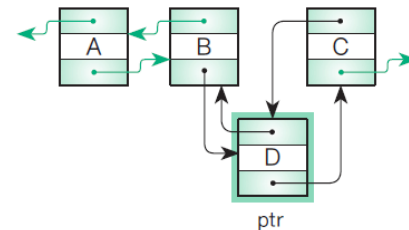
```
01  /*--- p가 가리키는 노드 바로 다음 노드를 삽입 ---*/
02  void InsertAfter(Dlist *list, Dnode *p, const Member *x)
03  {
04      Dnode *ptr = AllocDNode();
05      Dnode *nxt = p->next;
06      p->next = p->next->prev = ptr;
07      SetDNode(ptr, x, p, nxt);
08      list->crnt = ptr;      // 삽입한 노드를 선택
09  }
10
```

• 완성 파일 chap08/CircDblLinkedList.c

a 삽입 전



b 삽입 후



[그림 8-27] 원형 이중 연결 리스트에 노드를 삽입하는 과정



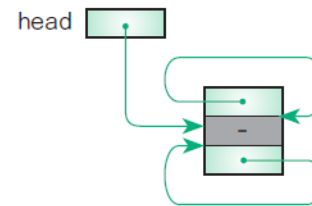
08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (15)

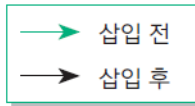
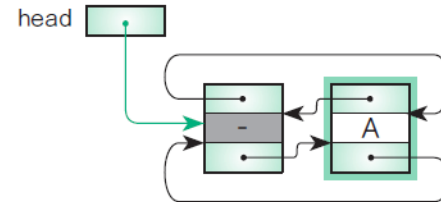
■ 바로 다음에 노드를 삽입하는 InsertAfter 함수 - (2)

- 리스트 머리에 더미 노드가 있어 '비어 있는 리스트에 삽입하는 경우'와 '리스트 머리에 삽입하는 경우'를 따로 처리하지 않아도 됨
- 그림 8-28에는 더미 노드만 있는 빈 리스트에 노드 A를 삽입하는데 삽입하기 전에 cnt, head는 모두 더미 노드를 가리키고 있음
- 삽입 과정
 - 1. 만든 노드의 앞쪽 포인터와 뒤쪽 포인터는 더미 노드를 가리킴
 - 2. 더미 노드의 뒤쪽 포인터와 앞쪽 포인터가 가리키는 노드는 A
 - 3. 선택한 노드가 가리키는 노드는 삽입한 노드

a 삽입 전



b 삽입 후



[그림8-28] 빈 원형 이중 연결 리스트에 노드를 삽입하는 과정



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (16)

- 머리에 노드를 삽입하는 **InsertFront** 함수
 - 머리 노드의 바로 뒤에 삽입
 - list->head가 가리키는 더미 노드 뒤에 노드를 삽입
- 꼬리에 노드를 삽입하는 **InsertRear** 함수
 - 꼬리 노드의 바로 뒤 = 더미 노드의 바로 앞'에 삽입
 - list->head->prev가 가리키는 꼬리 노드 뒤에 노드를 삽입

Do it! 실습 8-8[G]

• 완성 파일 chap08/CircDbllinkedList.c

```
01  /*--- 머리에 노드를 삽입 ---*/
02  void InsertFront(Dlist* list, const Member* x)
03  {
04      InsertAfter(list, list->head, x);
05  }
06
07  /*--- 꼬리에 노드를 삽입 ---*/
08  void InsertRear(Dlist* list, const Member* x)
09  {
10      InsertAfter(list, list->head->prev, x);
11  }
12
```

(실습 8-8[H]에서 계속)



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (17)

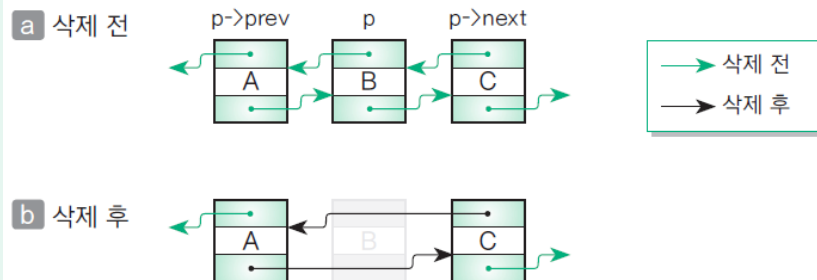
■ 노드를 삭제하는 Remove 함수

- 포인터 p가 가리키는 노드를 삭제하는 함수
- 삭제 과정
 - 1. 노드 A의 뒤쪽 포인터 p->prev->next가 가리키는 노드가 C(p->next)가 되도록 업데이트
 - 2. 노드 C의 앞쪽 포인터 p->next->prev가 가리키는 노드가 A(p->prev)가 되도록 업데이트하고 p가 가리키는 메모리 영역을 해제
 - 3. 선택한 노드가 삭제한 노드의 앞쪽 노드 A를 가리킬 수 있도록 cnt를 업데이트

Do it! 실습 8-8[H]

• 완성 파일 chap08/CircDbLinkedList.c

```
01  /*--- p가 가리키는 노드를 삭제---*/
02  void Remove(Dlist* list, Dnode* p)
03  {
04      p->prev->next = p->next;
05      p->next->prev = p->prev;
06      list->crnt = p->prev;    // 삭제한 노드의 앞쪽 노드를 선택
07      free(p);
08      if (list->crnt == list->head)
09          list->crnt = list->head->next;
10  }
11
```



[그림 8-29] 원형 이중 연결 리스트에서 노드를 삭제하는 과정

(실습 8-8[I]에서 계속)



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (18)

■ 머리 노드를 삭제하는 RemoveFront 함수

- Remove 함수를 사용해 포인터 list->head->next가 가리키는 머리 노드를 삭제
- 이때 더미 노드는 삭제하면 안 됨
- 따라서 list->head가 가리키는 더미 노드가 아닌 그 다음의 노드 list->head->next를 삭제

■ 꼬리 노드를 삭제하는 RemoveRear 함수

- Remove 함수를 사용해서 포인터 list->head->prev가 가리키는 꼬리 노드를 삭제

Do it! 실습 8-8[]

• 완성 파일 chap08/CircDblLinkedList.c

```
01  /*--- 머리 노드를 삭제 ---*/
02  void RemoveFront(Dlist* list)
03  {
04      if (!IsEmpty(list))
05          Remove(list, list->head->next);
06  }
07
08  /*--- 꼬리 노드를 삭제 ---*/
09  void RemoveRear(Dlist* list)
10  {
11      if (!IsEmpty(list))
12          Remove(list, list->head->prev);
13  }
```



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (19)

- 선택한 노드를 삭제하는 **RemoveCurrent** 함수
 - Remove 함수를 사용해서 포인터 list->crnt가 가리키는 노드를 삭제
- 모든 노드를 삭제하는 **Clear** 함수
 - 더미 노드를 제외하고 모든 노드를 삭제하는 함수
 - 리스트가 텅 빌 때까지 RemoveFront 함수를 사용해 머리 노드의 삭제를 반복
 - 선택한 포인터 list->crnt가 가리키는 노드는 더미 노드 list->head로 업데이트
- 원형 이중 연결 리스트를 종료하는 **Terminate** 함수
 - 먼저 Clear 함수를 호출해 모든 노드를 삭제하고 더미 노드의 메모리 영역도 해제

```
15  /*--- 선택한 노드를 삭제 ---*/
16  void RemoveCurrent(Dlist* list)

17  {
18      if (list->crnt != list->head)
19          Remove(list, list->crnt);
20  }

21
22  /*--- 모든 노드를 삭제 ---*/
23  void Clear(Dlist* list)
24  {
25      while (!IsEmpty(list))           // 텅 빌 때까지
26          RemoveFront(list);           // 머리 노드를 삭제
27  }

28
29  /*--- 원형 이중 연결 리스트 종료 ---*/
30  void Terminate(Dlist* list)
31  {
32      Clear(list);                       // 모든 노드를 삭제
33      free(list->head);                   // 더미 노드를 삭제
34  }
```



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (20)

- 실습 8-9 - (1)
 - 원형 이중 연결 리스트를 사용한 프로그램

Do it! 실습 8-9

• 완성 파일 chap08/CircDblLinkedListTest.c

```
01  /* 원형 이중 연결 리스트를 사용하는 프로그램 */
02  #include <stdio.h>
03  #include "Member.h" 실습 10-1에서 작성
04  #include "CircDblLinkedList.h"
05
06  /*--- 메뉴 ---*/
07  typedef enum {
08      TERMINATE, INS_FRONT, INS_REAR, RMV_FRONT, RMV_REAR, PRINT_CRNT,
09      RMV_CRNT, SRCH_NO, SRCH_NAME, PRINT_ALL, NEXT, PREV, CLEAR
10  } Menu;
```

```
12  /*--- 메뉴 선택 ---*/
13  Menu SelectMenu (void)
14  {
15      int ch;
16      char *mstring[] = {
17          "머리에 노드를 삽입",      "꼬리에 노드를 삽입",      "머리 노드를 삭제",
18          "꼬리 노드를 삭제",      "선택한 노드를 출력",      "선택한 노드를 삭제",
19          "번호로 검색",            "이름으로 검색",          "모든 노드를 출력",
20          "선택한 노드를 뒤쪽으로",  "선택한 노드를 앞쪽으로",  "모든 노드를 삭제",
21      };
22
23      do {
24          for(int i = TERMINATE; i < CLEAR; i++) {
25              printf("(%2d) %-22.22s ", i + 1, mstring[i]);
26              if((i % 3) == 2)
27                  putchar('\n');
28          }
29          printf("(0) 종료: ");
30          scanf("%d", &ch);
31      } while(ch < TERMINATE || ch > CLEAR);
32      return(Menu)ch;
33  }
```



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (21)

■ 실습 8-9 - (2)

```
34  /*--- 메인 ---*/
35  int main (void)

36  {
37      Menu menu;
38      Dlist list;
39      Initialize(&list);  // 원형 이중 연결 리스트를 초기화
40      do {
41          Member x;
42          switch(menu = SelectMenu()) {
43              /* 머리에 노드를 삽입 */
44              case INS_FRONT :
45                  x = ScanMember("머리에 삽입", MEMBER_NO | MEMBER_NAME);
46                  InsertFront(&list, &x);
47                  break;
48
49              /* 꼬리에 노드를 삽입 */
50              case INS_REAR :
51                  x = ScanMember("꼬리에 삽입", MEMBER_NO | MEMBER_NAME);
52                  InsertRear(&list, &x);
53                  break;
```

```
55          /* 머리 노드를 삭제 */
56          case RMV_FRONT :
57              RemoveFront(&list);
58              break;

60          /* 꼬리 노드를 삭제 */
61          case RMV_REAR :
62              RemoveRear(&list);
63              break;

64
65          /* 선택한 노드의 데이터를 출력 */
66          case PRINT_CRNT :
67              PrintLnCurrent(&list);
68              break;
69
70          /* 선택한 노드를 삭제 */
71          case RMV_CRNT :
72              RemoveCurrent(&list);
73              break;
```



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (22)

■ 실습 8-9 - (3)

```
75  /* 번호로 검색 */
76  case SRCH_NO :

77      x = ScanMember("검색", MEMBER_NO);
78      if(search(&list, &x, MemberNoCmp) != NULL)
79          PrintLnCurrent(&list);
80      else
81          puts("그 번호의 데이터가 없습니다.");
82      break;

83
84  /* 이름으로 검색 */
85  case SRCH_NAME :
86      x = ScanMember("검색", MEMBER_NAME);
87      if(search(&list, &x, MemberNameCmp) != NULL)
88          PrintLnCurrent(&list);
89      else
90          puts("그 이름의 데이터가 없습니다.");
91      break;

92
93  /* 모든 노드의 데이터를 출력 */
94  case PRINT_ALL :
95      Print(&list);
96      break;
```

```
98  /* 선택한 노드를 뒤쪽으로 진행 */
99  case NEXT :
100      Next(&list);
101      break;

102
103  /* 선택한 노드를 앞쪽으로 진행 */
104  case PREV :
105      Prev(&list);
106      break;

107
108  /* 모든 노드를 삭제 */
109  case CLEAR :
110      Clear(&list);
111      break;
112  }
113 } while(menu != TERMINATE);
114 Terminate(&list);    // 원형 이중 연결 리스트 종료
115
116 return 0;
117 }
```



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (23)

■ 실습 8-9 실행 결과 - (1)

실행 결과

(1) 머리에 노드를 삽입

(2) 꼬리에 노드를 삽입

(3) 머리 노드를 삭제

(4) 꼬리 노드를 삭제

(5) 선택한 노드를 출력

(6) 선택한 노드를 삭제

(7) 번호로 검색

(8) 이름으로 검색

(9) 모든 노드를 출력

(10) 선택한 노드를 뒤쪽으로

(11) 선택한 노드를 앞쪽으로

(12) 모든 노드를 삭제

(0) 종료: 1

머리에 삽입하는 데이터를 입력하세요.

번호: 1

이름: 모모

{1, 모모}를 머리에 삽입

(1) 머리에 노드를 삽입

(2) 꼬리에 노드를 삽입

(3) 머리 노드를 삭제

(4) 꼬리 노드를 삭제

(5) 선택한 노드를 출력

(6) 선택한 노드를 삭제

(7) 번호로 검색

(8) 이름으로 검색

(9) 모든 노드를 출력

(10) 선택한 노드를 뒤쪽으로

(11) 선택한 노드를 앞쪽으로

(12) 모든 노드를 삭제

(0) 종료: 2

꼬리에 삽입하는 데이터를 입력하세요.

번호: 5

이름: 나연

{5, 나연}을 꼬리에 삽입

(1) 머리에 노드를 삽입

(2) 꼬리에 노드를 삽입

(3) 머리 노드를 삭제

(4) 꼬리 노드를 삭제

(5) 선택한 노드를 출력

(6) 선택한 노드를 삭제

(7) 번호로 검색

(8) 이름으로 검색

(9) 모든 노드를 출력

(10) 선택한 노드를 뒤쪽으로

(11) 선택한 노드를 앞쪽으로

(12) 모든 노드를 삭제

(0) 종료: 1

머리에 삽입하는 데이터를 입력하세요.

번호: 10

이름: 정연

{10, 정연}을 머리에 삽입

- | | | |
|-------------------|-------------------|----------------|
| (1) 머리에 노드를 삽입 | (2) 꼬리에 노드를 삽입 | (3) 머리 노드를 삭제 |
| (4) 꼬리 노드를 삭제 | (5) 선택한 노드를 출력 | (6) 선택한 노드를 삭제 |
| (7) 번호로 검색 | (8) 이름으로 검색 | (9) 모든 노드를 출력 |
| (10) 선택한 노드를 뒤쪽으로 | (11) 선택한 노드를 앞쪽으로 | (12) 모든 노드를 삭제 |
- (0) 종료: 2

꼬리에 삽입하는 데이터를 입력하세요.

번호: 12 {12, 사나}를 꼬리에 삽입

이름: 사나

- | | | |
|-------------------|-------------------|----------------|
| (1) 머리에 노드를 삽입 | (2) 꼬리에 노드를 삽입 | (3) 머리 노드를 삭제 |
| (4) 꼬리 노드를 삭제 | (5) 선택한 노드를 출력 | (6) 선택한 노드를 삭제 |
| (7) 번호로 검색 | (8) 이름으로 검색 | (9) 모든 노드를 출력 |
| (10) 선택한 노드를 뒤쪽으로 | (11) 선택한 노드를 앞쪽으로 | (12) 모든 노드를 삭제 |
- (0) 종료: 1

머리에 삽입하는 데이터를 입력하세요.

번호: 14 {14, 지효}를 머리에 삽입

이름: 지효

- | | | |
|-------------------|-------------------|----------------|
| (1) 머리에 노드를 삽입 | (2) 꼬리에 노드를 삽입 | (3) 머리 노드를 삭제 |
| (4) 꼬리 노드를 삭제 | (5) 선택한 노드를 출력 | (6) 선택한 노드를 삭제 |
| (7) 번호로 검색 | (8) 이름으로 검색 | (9) 모든 노드를 출력 |
| (10) 선택한 노드를 뒤쪽으로 | (11) 선택한 노드를 앞쪽으로 | (12) 모든 노드를 삭제 |
- (0) 종료: 4

꼬리의 {12, 사나}를 삭제

- | | | |
|-------------------|-------------------|----------------|
| (1) 머리에 노드를 삽입 | (2) 꼬리에 노드를 삽입 | (3) 머리 노드를 삭제 |
| (4) 꼬리 노드를 삭제 | (5) 선택한 노드를 출력 | (6) 선택한 노드를 삭제 |
| (7) 번호로 검색 | (8) 이름으로 검색 | (9) 모든 노드를 출력 |
| (10) 선택한 노드를 뒤쪽으로 | (11) 선택한 노드를 앞쪽으로 | (12) 모든 노드를 삭제 |
- (0) 종료: 8

검색하는 데이터를 입력하세요.

이름: 사나 {사나} 검색 실패

그 이름의 데이터가 없습니다.



08-4 원형 이중 연결 리스트

원형 이중 연결 리스트 만들기 - (24)

■ 실습 8-9 실행 결과 - (2)

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 선택한 노드를 뒤쪽으로 (11) 선택한 노드를 앞쪽으로 (12) 모든 노드를 삭제
(0) 종료: 7

(0) 종료: 7

검색하는 데이터를 입력하세요.

번호: 10

{10} 검색 성공

10 정연

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 선택한 노드를 뒤쪽으로 (11) 선택한 노드를 앞쪽으로 (12) 모든 노드를 삭제
(0) 종료: 5

10 정연

선택한 노드는 (10, 정연)

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 선택한 노드를 뒤쪽으로 (11) 선택한 노드를 앞쪽으로 (12) 모든 노드를 삭제
(0) 종료: 11

선택 노드를 앞쪽으로

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 선택한 노드를 뒤쪽으로 (11) 선택한 노드를 앞쪽으로 (12) 모든 노드를 삭제
(0) 종료: 5

14 지효

선택한 노드는 (14, 지효)

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 선택한 노드를 뒤쪽으로 (11) 선택한 노드를 앞쪽으로 (12) 모든 노드를 삭제
(0) 종료: 9

【 모두 보기 】

14 지효

10 정연

1 모모

5 나연

모든 노드를 순서대로 출력

(1) 머리에 노드를 삽입 (2) 꼬리에 노드를 삽입 (3) 머리 노드를 삭제
(4) 꼬리 노드를 삭제 (5) 선택한 노드를 출력 (6) 선택한 노드를 삭제
(7) 번호로 검색 (8) 이름으로 검색 (9) 모든 노드를 출력
(10) 선택한 노드를 뒤쪽으로 (11) 선택한 노드를 앞쪽으로 (12) 모든 노드를 삭제
(0) 종료: 0